PROGRAMACIÓN II · UNIDAD 1 · CLASE 7

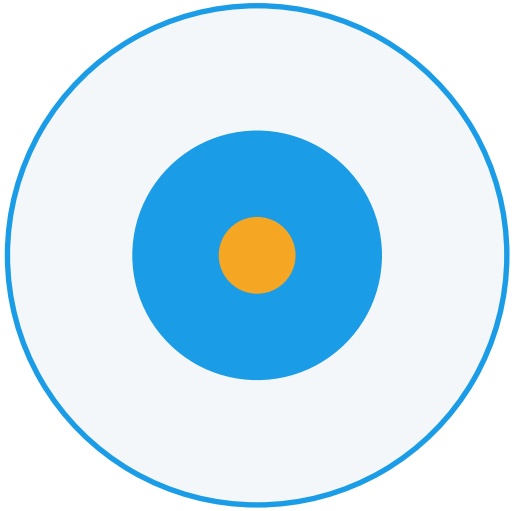
Polimorfismo y colecciones: List

Bloque 4 · Programación Orientada a Objetos en C#

> *Consolidar el polimorfismo como síntesis de override, interfaces y composición, e introducir List.*

Objetivo de la clase

Consolidar el **polimorfismo** como síntesis de override, interfaces y composición, e introducir la colección `List`.



Qué veremos hoy

1

Polimorfismo

2

Colecciones vs. arrays

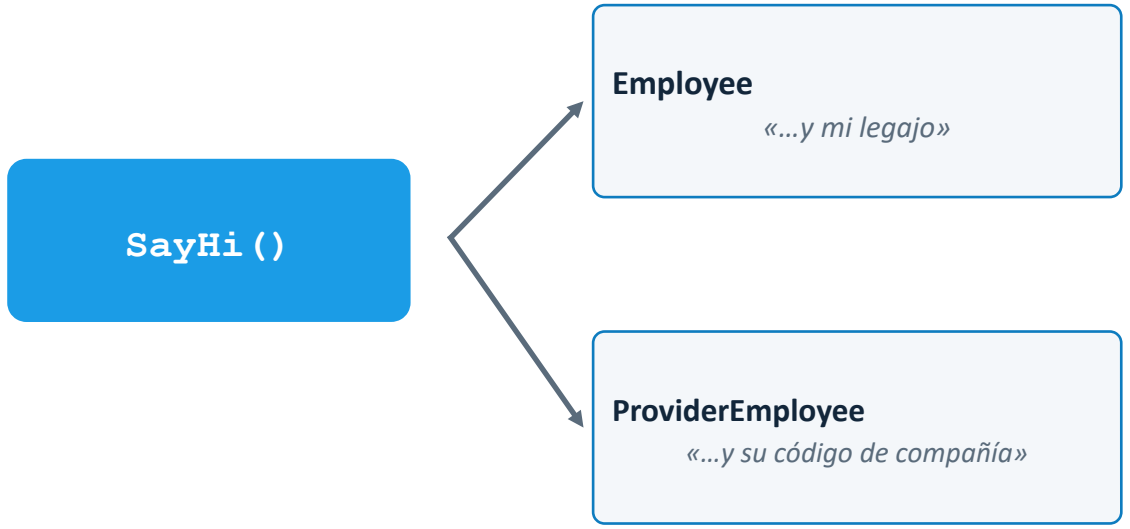
3

La colección List<T>

1 Polimorfismo: mismo llamado, distintas respuestas

Un mismo llamado produce distintas respuestas, según el tipo real del objeto que lo recibe.

Es la síntesis de tres recursos ya vistos: la sobreescritura, las interfaces y la composición.



Apunte Unidad 1, pág. 35

1 Una síntesis de lo ya visto

Sobreescritura

override redefine el mismo método en cada subclase.

Interfaces

Tratar objetos por su contrato (ITurnable), sin importar su clase.

Composición

Reasignar componentes cambia el comportamiento en tiempo de ejecución.

POLIMORFISMO *mismo llamado, distintas respuestas*

Apunte Unidad 1, pág. 35

2 Colecciones vs. arrays

Array

Tamaño fijo, definido al crear. Para crecer hay que crear otro array más grande y copiar los elementos.

Colección

Se autodimensiona: crece y se reduce dinámicamente durante la ejecución, a cambio de un mayor costo en recursos.

Viven en el namespace `System.Collections.Generic` y son genéricas: se tipan con `<T>`.

Apunte Unidad 1, pág. 35-36

3 La colección List<T>

List.cs

```
using System.Collections.Generic;

List<Person> people = new List<Person>();

people.Add(new Employee("Ada"));          // Add
people.Add(new ProviderEmployee("Alan"));

people.RemoveAt(0);                       // RemoveAt (índice)

foreach (Person p in people)              // foreach
{
    Console.WriteLine(p.Describe());
}
```

- Creación: List<T>, con el tipo entre < >.
- Add: agrega un elemento al final.
- RemoveAt: elimina por índice.
- foreach: recorre todos los elementos.

Apunte Unidad 1, pág. 36-37

3 Polimorfismo en acción sobre List<T>

Program.cs

```
List<Person> people = new List<Person>();
people.Add(new Employee("Ada"));
people.Add(new ProviderEmployee("Alan"));

foreach (Person p in people)
{
    p.SayHi();      // mismo llamado...
}
```

Salida

```
Hi, I'm Ada · legajo 1024
Hi, I'm Alan · código de compañía PRV-7
```

(salidas ilustrativas)

La lista se recorre como `Person` (genérico), pero cada objeto responde según su tipo real: eso es el polimorfismo.

Apunte Unidad 1, pág. 36-37

EN SÍNTESIS

- Polimorfismo: un mismo llamado, distintas respuestas según el tipo real del objeto.
- Es la síntesis de la sobreescritura, las interfaces y la composición.
- Colecciones vs. arrays: las colecciones se autodimensionan (con más costo en recursos).
- List<T> vive en System.Collections.Generic; se crea, se usa Add / RemoveAt y se recorre con foreach.
- Recorrer una List<Person> y llamar SayHi() muestra el polimorfismo en acción.

Próxima clase (práctica): polimorfismo en acción operando sobre listas (alta, baja, recorrido y búsqueda con List<T>).